AppSec Requirements — OWASP Juice Shop

▲ **DEMO catalog** — audited against the packaged example requirements, not your organization's. Configure a real source with --requirements / an org profile.

Field	Value
Generated	2026-06-13
Repository	OWASP Juice Shop
Source	packaged example (DEMO)
Open Requirements	36
🔴 Failed	28
🟤 Partial	8
🟢 Passed	5
⚪ Unverifiable	10
➖ Not applicable	13

Open Requirements

> 🔴 fail · 🟤 partial — open gaps detailed below. 🟢 passed, ⚪ unverifiable, and ➖ not-applicable are counted in the summary only.

Status	Priority	ID	Requirement	Effort
🔴 FAIL	MUST	AC-001	No Mutual API Authentication	L
🔴 FAIL	MUST	AC-002	No RBAC / Deny-By-Default	L
🔴 FAIL	MUST	AC-004	No Central IdP / Mandatory MFA	L
🔴 FAIL	MUST	AC-006	IDOR / No Resource-Level Authz	L
🔴 FAIL	MUST	DP-001	Data Not Encrypted At Rest	L
🔴 FAIL	MUST	DP-002	Weak / Hardcoded Keys	M
🔴 FAIL	MUST	DP-004	User Passwords Stored (MD5)	L
🔴 FAIL	MUST	DP-005	Secrets Not In A Secret Manager	M
🔴 FAIL	MUST	EH-001	Stack Traces Disclosed On Error	S
🔴 FAIL	MUST	EH-002	Detailed Errors Returned To Clients	S
🔴 FAIL	MUST	HN-002	Management Endpoints Public	S
🔴 FAIL	MUST	HN-003	Default Accounts / Unused Features Enabled	M
🔴 FAIL	MUST	IF-001	No Container Image Vulnerability Scan	S
🔴 FAIL	MUST	IV-001	Input Not Restrictively Validated	L
🔴 FAIL	MUST	IV-002	XML Parser Allows External Entities	S
🔴 FAIL	MUST	IV-003	Unsafe Deserialization Of Uploads	M
🔴 FAIL	MUST	IV-004	Raw String-Interpolated SQL	M
🔴 FAIL	MUST	IV-005	Insecure Upload Handling	M
🔴 FAIL	MUST	LM-002	Logs Not Structured	S
🔴 FAIL	MUST	SC-001	No SCA In CI	S
🔴 FAIL	MUST	SC-002	Dependencies Not Pinned / No Lockfile	M
🔴 FAIL	MUST	WEB-001	Missing Anti-CSRF Controls	M
🔴 FAIL	MUST	WEB-002	Session Token In localStorage	M
🔴 FAIL	MUST	WEB-003	Wildcard CORS	S
🔴 FAIL	MUST	WEB-005	Missing HSTS Header	S
🔴 FAIL	MUST	WEB-007	Unsafe HTML Sink Bypass	M

Status	Priority	ID	Requirement	Effort
● FAIL	SHOULD	IF-007	Base Image Not Pinned To Digest	S
● FAIL	SHOULD	WEB-004	No Content Security Policy	M
● PARTIAL	MUST	AC-003	Rate Limiting Incomplete	S
● PARTIAL	MUST	AC-005	Token Claims Partly Validated	M
● PARTIAL	MUST	DP-003	Some Sensitive Data On GET	M
● PARTIAL	MUST	EH-003	Inconsistent Auth Failure Handling	M
● PARTIAL	MUST	HN-001	Server Header Not Masked	S
● PARTIAL	MUST	IV-006	Payload Limits Not Enforced	M
● PARTIAL	MUST	LM-001	Partial Security-Event Logging	M
● PARTIAL	SHOULD	WEB-008	Missing Referrer/Permissions-Policy	S

● FAIL · MUST · AC-001 — No Mutual API Authentication

> **Requirement:** All API-to-API calls MUST use mutual authentication via a standard mechanism (e.g. OAuth 2.0 client credentials, mTLS)

Service authentication relies on a self-signed RS256 JWT whose private key is hardcoded in `lib/insecurity.ts:23`; there is no mTLS or OAuth client-credentials flow.

Evidence: `lib/insecurity.ts:23`, `lib/insecurity.ts:56`

Risk: Anyone with the embedded key mints valid tokens for any service-to-service call.

Fix: Use a standard mutual mechanism (OAuth2 client-credentials or mTLS) as the requirement demands; stop signing with an in-source key.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
- Blueprint: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

● FAIL · MUST · AC-002 — No RBAC / Deny-By-Default

> **Requirement:** Apply least-privilege and role-based access control (RBAC) to all endpoints; deny by default

Endpoints lack a central deny-by-default authorization layer, and access is not role-scoped — broken access control is pervasive (IDOR across baskets, orders, and user data).

Evidence: `routes/` (no central authorization middleware)

Risk: Vertical and horizontal privilege escalation across users and admin functions.

Fix: Add deny-by-default RBAC middleware and per-endpoint role checks, as the Access Control cheat sheet prescribes — not merely an `isAuthorized()` presence check.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html

● FAIL · MUST · AC-004 — No Central IdP / Mandatory MFA

> **Requirement:** End users MUST be authenticated through a central identity provider using a standard protocol (OIDC/SAML) with mandatory MFA

The app implements a custom email/password login in `routes/login.ts`; there is no OIDC/SAML identity provider and MFA is optional.

Evidence: `routes/login.ts:34`, `lib/insecurity.ts:56`

Risk: Credential-based account takeover without federated step-up authentication.

Fix: Authenticate via an OIDC/SAML central IdP with mandatory MFA, replacing the custom login.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Multifactor_Authentication_Cheat_Sheet.html
- Blueprint: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

● FAIL · MUST · AC-006 — IDOR / No Resource-Level Authz

> **Requirement:** Enforce resource-level authorization checks to prevent Insecure Direct Object Reference (IDOR); compare token identity claims against the requested resource

Object access is not ownership-scoped — baskets, orders, and feedback resolve by id without comparing the caller's identity claim against the resource owner.

Evidence: `routes/` (no ownership comparison)

Risk: Users read and modify other users' resources by changing the object id.

Fix: Compare the JWT identity claim against the requested resource's owner on every object access, per the IDOR Prevention cheat sheet.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

● FAIL · MUST · DP-001 — Data Not Encrypted At Rest

> **Requirement:** Encrypt confidential data at rest using approved algorithms (AES-256-GCM or equivalent); apply full-disk or database-level encryption for all production data stores

The SQLite database and uploaded files persist in plaintext; there is no database- or disk-level encryption.

Evidence: `models/`, `uploads/`

Risk: Disk or backup access exposes user PII and credentials directly.

Fix: Enable database-/disk-level AES-256 encryption for production stores, as the requirement demands.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

● FAIL · MUST · DP-002 — Weak / Hardcoded Keys

> **Requirement:** Security tokens, API keys, and cryptographic keys MUST be generated using a cryptographically secure random source and be at least 256 bits (32 bytes) of entropy

A 24-character HMAC secret is hardcoded at `lib/insecurity.ts:44` and a 1024-bit RSA private key is embedded at `lib/insecurity.ts:23` — both public in source and below the 256-bit bar.

Evidence: `lib/insecurity.ts:44`, `lib/insecurity.ts:23`

Risk: Keys are publicly readable and under-strength — tokens are forgeable.

Fix: Generate ≥256-bit keys from a CSPRNG at deploy time and load them from a secret manager, never from source.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

● FAIL · MUST · DP-004 — User Passwords Stored (MD5)

> **Requirement:** Do not store user credentials (passwords); delegate authentication to a central identity service

User credentials are stored locally and hashed with MD5 at `lib/insecurity.ts:43`.

Evidence: `lib/insecurity.ts:43`

Risk: Trivially crackable hashes on database compromise.

Fix: Delegate authentication to a central identity service rather than storing passwords, as the requirement directs.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

● FAIL · MUST · DP-005 — Secrets Not In A Secret Manager

> **Requirement:** Secrets (passwords, API keys, certificates) MUST be stored in a dedicated secret manager; rotate them regularly and on suspected compromise

The RSA private key and HMAC secret are committed in `lib/insecurity.ts`; there is no managed storage or rotation.

Evidence: `lib/insecurity.ts:23`, `lib/insecurity.ts:44`

Risk: Credential compromise via repo read; a leaked key remains valid indefinitely.

Fix: Move secrets to a dedicated secret manager with scheduled rotation, per the Secrets Management cheat sheet.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

● FAIL · MUST · EH-001 — Stack Traces Disclosed On Error

> **Requirement:** On any error (expected or unexpected) the application MUST remain in a secure state and MUST NOT disclose internal details (stack traces, SQL, paths) to clients

The Express `errorhandler()` middleware at `server.ts:676` returns full stack traces, and `errorhandler.title` leaks the Express version at `server.ts:719`.

Evidence: `server.ts:676`, `server.ts:719`

Risk: Internal paths, SQL, and stack frames are disclosed to users.

Fix: Restrict `errorhandler` to development; in production keep a secure state and emit no internal detail.

```
// Before
app.use(errorhandler())

// After
if (process.env.NODE_ENV !== 'production') app.use(errorhandler())
else app.use((err, req, res, next) => { logger.error(err); res.status(500).json({ error: 'Internal
        error' }) })
```

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

● FAIL · MUST · EH-002 — Detailed Errors Returned To Clients

> **Requirement:** Return generic error messages to clients; log detailed diagnostics server-side only

The development `errorhandler()` returns detailed diagnostics in the HTTP response (`server.ts:676`).

Evidence: `server.ts:676`

Risk: Diagnostic detail is exposed to anonymous clients.

Fix: Return a generic error body to clients and log details server-side only, as the requirement states.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

● FAIL · MUST · HN-002 — Management Endpoints Public

> **Requirement:** Sensitive management and debug endpoints MUST NOT be reachable from public networks; serve them on a dedicated internal-only interface or port

`/metrics` (Prometheus) is served unauthenticated at `server.ts:718` and Swagger `/api-docs` at `server.ts:286`.

Evidence: `server.ts:718`, `server.ts:286`

Risk: Operational surface and the full API schema are reachable by anonymous users.

Fix: Move `/metrics` and `/api-docs` to a dedicated internal-only interface or port.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

● FAIL · MUST · HN-003 — Default Accounts / Unused Features Enabled

> **Requirement:** Disable all unused features, modules, and default accounts before deploying to production

The app seeds default/demo accounts (including admin) and ships extensive demo functionality enabled by default.

Evidence: `data/static/users.yml`, data seeding

Risk: Known default credentials and surplus features widen the attack surface.

Fix: Remove default/seed accounts and disable unused modules before production, per A05 Security Misconfiguration.

Effort: M

Links:

- Requirement: https://owasp.org/Top10/A05_2021-Security_Misconfiguration/

● FAIL · MUST · IF-001 — No Container Image Vulnerability Scan

> **Requirement:** Container images MUST be scanned for known vulnerabilities before deployment; block promotion of images with critical or high vulnerabilities that have a fix available

There is no image scanner (Trivy/Grype) in `.github/workflows/`; the image is built and shipped unscanned.

Evidence: `.github/workflows/` (no image-scan job)

Risk: Vulnerable OS/runtime layers ship to production undetected.

Fix: Add a Trivy/Grype scan gate to CI that blocks critical/high fixable findings, per the Docker Security cheat sheet.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

● FAIL · MUST · IV-001 — Input Not Restrictively Validated

> **Requirement:** Validate all external input as restrictively as possible; reject unknown fields rather than ignoring them (allowlist over denylist)

There is no schema/allowlist validation layer; raw `req.body` / `req.query` flow into SQL, file paths, and parsers.

Evidence: `routes/search.ts:23` , `routes/fileUpload.ts:42`

Risk: Injection and traversal across multiple sinks from unvalidated input.

Fix: Add allowlist schema validation per route (`express-validator` / `zod`) that rejects unknown fields before input reaches the sinks.

Effort: L

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html

● FAIL · MUST · IV-002 — XML Parser Allows External Entities

> **Requirement:** Harden XML parsers to prevent XXE attacks; disable external entity resolution and DTD processing

`libxml.parseXml(data, { noent: true })` at `routes/fileUpload.ts:83` resolves external entities (XXE).

Evidence: `routes/fileUpload.ts:83`

Risk: File disclosure / SSRF via a crafted DOCTYPE in uploaded complaint XML.

Fix: Remove `noent: true` (default is false) and disable DTD loading.

```
// Before
libxml.parseXml(data, { noblanks: true, noent: true, nocdata: true })

// After
libxml.parseXml(data, { noblanks: true, nocdata: true }) // noent defaults to false; DTD/entities not
        resolved
```

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html

● FAIL · MUST · IV-003 — Unsafe Deserialization Of Uploads

> **Requirement:** Restrict and validate object graphs during deserialization; never deserialize untrusted data into privileged types

`yaml.load()` runs on attacker-uploaded files inside a `vm` sandbox at `routes/fileUpload.ts:117`, and `notevil` evaluates B2B order data at `routes/b2bOrder.ts:9`.

Evidence: `routes/fileUpload.ts:117`, `routes/b2bOrder.ts:9`

Risk: YAML-bomb DoS and sandbox-escape attempts from untrusted upload content.

Fix: Restrict and validate before deserializing untrusted documents (cap size/depth) and drop the `vm` + `notevil` eval path, per the Deserialization cheat sheet.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Deserialization_Cheat_Sheet.html

● FAIL · MUST · IV-004 — Raw String-Interpolated SQL

> **Requirement:** Use parameterized queries, prepared statements, or ORM methods for all database access; never concatenate user input into query strings

User input is concatenated into `sequelize.query()` at `routes/login.ts:34` and `routes/search.ts:23`.

Evidence: `routes/login.ts:34`, `routes/search.ts:23`

Risk: Authentication bypass and UNION-based data exfiltration via SQL injection.

Fix: Use bound replacements or ORM finder methods; never concatenate input.

```
// Before
models.sequelize.query(`SELECT * FROM Products WHERE name LIKE '%${criteria}%'`)

// After
models.sequelize.query('SELECT * FROM Products WHERE name LIKE :criteria',
  { replacements: { criteria: `%${criteria}%` } })
```

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

● FAIL · MUST · IV-005 — Insecure Upload Handling

> **Requirement:** Validate, sanitize, and securely store uploaded files; enforce content-type verification, size limits, and safe storage paths

The complaint upload writes archive entries to `uploads/complaints/ + fileName` without path containment (`routes/fileUpload.ts:42-45`) — a zip-slip — and then parses XML/YAML from the content.

Evidence: `routes/fileUpload.ts:42`, `routes/fileUpload.ts:45`

Risk: Path-traversal write outside the upload directory; XXE/deserialization from file content.

Fix: Verify each entry path resolves under the upload root and enforce content-type + size limits before parsing, per the File Upload cheat sheet.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

● FAIL · MUST · LM-002 — Logs Not Structured

> **Requirement:** Use structured log formats (e.g. JSON) with consistent fields (timestamp, request-id, user-id, action, outcome); avoid free-text-only log entries for security events

The winston logger uses `format.simple()` (free text) at `lib/logger.ts:12`, not structured JSON.

Evidence: `lib/logger.ts:12`

Risk: Security events are hard to parse, correlate, and alert on.

Fix: Switch the winston format to JSON with consistent fields.

```
// Before
format: winston.format.simple()

// After
format: winston.format.combine(winston.format.timestamp(), winston.format.json())
```

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

● FAIL · MUST · SC-001 — No SCA In CI

> **Requirement:** Integrate automated Software Composition Analysis (SCA) into the CI/CD pipeline; block builds on critical or high vulnerabilities with no fix available

CI runs CodeQL (SAST) and ZAP (DAST) but no SCA scan; `bom.json` generation alone is not scanning.

Evidence: `.github/workflows/` (no SCA job)

Risk: Known-vulnerable dependencies ship undetected.

Fix: Add an SCA scan (e.g. OWASP Dependency-Check) that blocks the build on critical/high findings.

Effort: S

Links:

- Requirement: https://owasp.org/www-project-dependency-check/

● FAIL · MUST · SC-002 — Dependencies Not Pinned / No Lockfile

> **Requirement:** Pin direct dependencies to exact versions and verify integrity (e.g. lockfiles, checksums, or hash pinning); do not use floating version ranges in production builds

`.npmrc` sets `package-lock=false` and no `package-lock.json` is committed; `package.json` uses floating `^` ranges.

Evidence: `.npmrc`, `package.json`

Risk: Non-reproducible builds and unverified, drifting transitive dependencies.

Fix: Commit a lockfile (remove `package-lock=false`) and pin direct dependencies to exact versions.

Effort: M

Links:

- Requirement: https://owasp.org/www-project-dependency-check/
- Blueprint: https://cheatsheetseries.owasp.org/cheatsheets/Third_Party_Javascript_Management_Cheat_Sheet.html

● FAIL · MUST · WEB-001 — Missing Anti-CSRF Controls

> **Requirement:** Implement Anti-CSRF protection (e.g. via SameSite cookies) for user sessions. State-changing actions MUST NOT be performed via HTTP GET

There is no SameSite cookie attribute or CSRF token enforcement, and CORS is wide-open at `server.ts:182`, so cross-origin state changes are not blocked.

Evidence: `server.ts:182`

Risk: Cross-site request forgery against authenticated, state-changing endpoints.

Fix: Set `SameSite=Strict/Lax` on the session cookie and add CSRF tokens, keeping state changes off HTTP GET.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

● FAIL · MUST · WEB-002 — Session Token In localStorage

> **Requirement:** Do not store sensitive data in any client-side storage mechanism (localStorage, sessionStorage, IndexedDB, cookies without HttpOnly)

The JWT auth token is persisted in `localStorage` at `frontend/src/app/app.guard.ts:18,35`.

Evidence: `frontend/src/app/app.guard.ts:18`, `frontend/src/app/app.guard.ts:35`

Risk: Any XSS payload can exfiltrate the bearer token; there is no HttpOnly protection.

Fix: Hold the session in an HttpOnly, Secure, SameSite cookie via a BFF; keep it out of web storage.

```
// Before
localStorage.setItem('token', token)

// After
// server sets: Set-Cookie: session=<jwt>; HttpOnly; Secure; SameSite=Lax
// client no longer reads or stores the token
```

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html

● FAIL · MUST · WEB-003 — Wildcard CORS

> **Requirement:** Configure CORS restrictively; use an explicit allowlist of trusted origins and never use wildcard (*) for authenticated APIs

`app.use(cors())` is applied with no origin allowlist at `server.ts:182`; the comment even labels it "Allow everything!".

Evidence: `server.ts:181`, `server.ts:182`

Risk: Any origin can issue credentialed cross-origin requests to the API.

Fix: Configure an explicit origin allowlist; never wildcard for authenticated APIs.

```
// Before
app.use(cors())

// After
app.use(cors({ origin: ['https://app.example.com'], credentials: true }))
```

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/CORS_Security_Cheat_Sheet.html

● FAIL · MUST · WEB-005 — Missing HSTS Header

> **Requirement:** Set the Strict-Transport-Security (HSTS) response header on all internet-facing services with a minimum max-age of one year

helmet supplies `noSniff` / `frameguard` but `helmet.hsts()` is never applied (`server.ts:185-188`).

Evidence: `server.ts:185`

Risk: No browser enforcement of HTTPS; vulnerable to SSL-strip downgrade.

Fix: Add the HSTS header with a one-year minimum max-age.

```
app.use(helmet.hsts({ maxAge: 31536000, includeSubDomains: true }))
```

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Strict_Transport_Security_Cheat_Sheet.html

● FAIL · MUST · WEB-007 — Unsafe HTML Sink Bypass

> **Requirement:** Encode all user-controlled output at the frontend to prevent Cross-Site Scripting (XSS); use framework-native binding rather than raw innerHTML

`bypassSecurityTrustHtml()` is applied to user-controlled values at `administration.component.ts:60,78` and `track-result.component.ts:48`.

Evidence: `frontend/src/app/administration/administration.component.ts:60`, `frontend/src/app/track-result/track-result.component.ts:48`

Risk: Stored/reflected XSS — Angular's built-in sanitizer is explicitly bypassed.

Fix: Remove `bypassSecurityTrust*` and rely on framework-native interpolation binding so Angular auto-encodes.

```
// Before
this.results.orderNo = this.sanitizer.bypassSecurityTrustHtml(`<code>${results.data[0].orderId}</code>`)

// After
this.results.orderNo = results.data[0].orderId // bound via {{ }} — Angular encodes it
```

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- Blueprint: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

● FAIL · SHOULD · IF-007 — Base Image Not Pinned To Digest

> **Requirement:** Pin base images to an immutable digest (SHA256) rather than a mutable tag; rebuild images regularly to incorporate upstream security patches

The Dockerfile pins by mutable tag (`FROM node:24`, `gcr.io/distroless/nodejs24-debian13`), not a SHA256 digest.

Evidence: `Dockerfile:1`, `Dockerfile:23`

Risk: The base image can change under the same tag, undermining reproducibility and provenance.

Fix: Pin each `FROM` to an immutable `@sha256:…` digest and rebuild regularly, per the Docker Security cheat sheet.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

● FAIL · SHOULD · WEB-004 — No Content Security Policy

> **Requirement:** Activate Content-Security-Policy (CSP) headers on all internet-facing applications; avoid unsafe-inline and unsafe-eval directives

`helmet.contentSecurityPolicy()` is not configured and there is no CSP meta in `frontend/src/index.html`.

Evidence: `server.ts:185`

Risk: No defense-in-depth against injected script execution.

Fix: Add a restrictive `helmet.contentSecurityPolicy({ directives })` without `unsafe-inline` / `unsafe-eval`.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

● PARTIAL · MUST · AC-003 — Rate Limiting Incomplete

> **Requirement:** Apply rate limiting to all externally reachable API endpoints to prevent brute-force and scraping attacks

`rateLimit` covers `/rest/user/reset-password` and a few endpoints (`server.ts:343,458-471`) but not `/rest/user/login` (`server.ts:594`) or most external APIs.

Evidence: `server.ts:594` , `server.ts:343`

Risk: Credential brute-force and scraping on unthrottled routes.

Fix: Extend the existing `express-rate-limit` to all externally reachable endpoints, including login.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html

● PARTIAL · MUST · AC-005 — Token Claims Partly Validated

> **Requirement:** Validate OAuth token claims (issuer, audience, expiry) on every request; do not cache authorization decisions beyond the token TTL

`expressJwt` verifies the signature and expiry, but issuer/audience claims are neither set nor validated (`lib/insecurity.ts:54,191`).

Evidence: `lib/insecurity.ts:54` , `lib/insecurity.ts:191`

Risk: A token from any issuer holding the (public) key is accepted.

Fix: Set and validate issuer + audience alongside expiry on every request, per the JWT cheat sheet.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

● PARTIAL · MUST · DP-003 — Some Sensitive Data On GET

> **Requirement:** Transmit sensitive data in the HTTP request body (POST/PUT) rather than in URLs or query parameters

Login/password operations use POST, but several sensitive lookups and state reads flow through GET query strings.

Evidence: `routes/` (GET handlers carrying sensitive params)

Risk: Sensitive values captured in logs, proxies, and browser history.

Fix: Move the remaining sensitive parameters into POST/PUT request bodies.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
- Blueprint: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

● PARTIAL · MUST · EH-003 — Inconsistent Auth Failure Handling

> **Requirement:** Handle authentication and authorization failures consistently to avoid information disclosure through timing or response differences

Login returns a generic credential error, but other endpoints surface differentiated 401/403/500 responses and error detail across handlers.

Evidence: `routes/login.ts`, `server.ts:676`

Risk: Response and timing differences enable user enumeration and probing.

Fix: Normalise authn/authz failure responses (status, body, and timing) across handlers, per the Authentication cheat sheet.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

● PARTIAL · MUST · HN-001 — Server Header Not Masked

> **Requirement:** Disable technology-disclosure headers (X-Powered-By, Server) or replace them with a non-descriptive value

`x-powered-by` is disabled (`server.ts:188`) but the Express `Server` header is unmasked and `errorhandler.title` leaks the Express version (`server.ts:719`).

Evidence: `server.ts:188`, `server.ts:719`

Risk: Framework/version fingerprinting aids targeted exploits.

Fix: Also strip or replace the `Server` header and remove the version from the error title.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html

● PARTIAL · MUST · IV-006 — Payload Limits Not Enforced

> **Requirement:** Apply strict limits on payload size, field length, array length, and JSON nesting depth to prevent resource exhaustion

Upload size is only checked to solve a challenge (`routes/fileUpload.ts:62`), not enforced; no global body-size/field/depth limits are set.

Evidence: `routes/fileUpload.ts:62`

Risk: Oversized or deeply-nested payloads can exhaust memory/CPU.

Fix: Set explicit body-parser size limits and field/array/depth caps, per the Denial of Service cheat sheet.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Denial_of_Service_Cheat_Sheet.html

● PARTIAL · MUST · LM-001 — Partial Security-Event Logging

> **Requirement:** Log all security-relevant events (authentication success/failure, authorization failure, privilege escalation, configuration changes) with sufficient context to support incident investigation

winston and morgan access logging exist (`lib/logger.ts`, `server.ts:338`), but authn/authz failures and privilege changes are not systematically logged.

Evidence: `lib/logger.ts:8`, `server.ts:338`

Risk: Insufficient trail to investigate auth abuse or escalation.

Fix: Emit structured security-event logs for auth success/failure, authorization denial, and configuration changes with request context.

Effort: M

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- Blueprint: https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

● PARTIAL · SHOULD · WEB-008 — Missing Referrer/Permissions-Policy

> **Requirement:** Set Referrer-Policy and Permissions-Policy response headers to limit information leakage and browser feature exposure

A `feature-policy` (payment) header is set (`server.ts:189`) but there is no `Referrer-Policy` and no modern `Permissions-Policy`.

Evidence: `server.ts:189`

Risk: Referrer leakage and broad browser-feature exposure.

Fix: Add `Referrer-Policy` and a modern `Permissions-Policy` header (via helmet), per the HTTP Headers cheat sheet.

Effort: S

Links:

- Requirement: https://cheatsheetseries.owasp.org/cheatsheets/HTTP_Headers_Cheat_Sheet.html

---

Effort: S = under 1 hour · M = about half a day · L = multi-day or architectural change.